M04 — Randomization & Probabilistic Analysis

Sources: CLRS Ch. 5 (Probabilistic Analysis and Randomized Algorithms) + App. C · Skiena §4.6.2 (Randomized Algorithms), Ch. 6 §§6.1–6.3, 6.8–6.10

Big Idea

Worst-case analysis is pessimistic and average-case analysis requires you to assume an input distribution you usually don't know. **Randomization escapes both.** By making the *algorithm* flip coins rather than assuming the *input* is random, you convert "there exists a bad input" into "there exist unlucky coin flips" — and no adversary can construct unlucky coin flips. That is the whole idea: *randomization makes the worst-case input go away.* The analytical machinery is small and reusable: **indicator random variables** plus **linearity of expectation**, which holds *even when the variables are dependent*, and which turns almost every counting-expectation problem into a one-line sum. The two structural facts to carry: `Σ1/i = H_n ≈ ln n` (hiring, coupon collector, quicksort) and the balls-and-bins constants (`1/e ≈ 36.8%` of bins empty, max load `Θ(log n / log log n)`). And remember the vocabulary distinction — **average-case** is expectation over inputs, **expected** is expectation over the algorithm's own coin flips.

What You Should Be Able To Do After This Chapter

- State the difference between **probabilistic analysis** and a **randomized algorithm**, and between **average-case** and **expected** running time — and use the words correctly.
- Define an indicator random variable for an event and apply **linearity of expectation** to sum them, including when the events are dependent.
- Analyze the hiring problem, the birthday paradox, balls-and-bins, coupon collector, streaks, and the secretary problem — each in a few lines.
- Write **Fisher–Yates** shuffle correctly, and explain why the "swap with a random element anywhere" variant is wrong.
- Prove Fisher–Yates uniform via its `k`-permutation loop invariant.
- Distinguish **Las Vegas** from **Monte Carlo** algorithms and say which guarantee each gives up.

- Explain why hashing needs a *randomly chosen* hash function to earn a randomized guarantee, and construct such a family.
- Explain Fermat/Miller–Rabin primality testing and why Carmichael numbers matter.
- Say what actually happens when randomized quicksort "hits its worst case" (nothing — the distribution is extremely tight).

---

1. Two different uses of probability

The distinction, stated precisely

This is the vocabulary CLRS insists on, and getting it wrong in an interview is a tell.

	PROBABILISTIC ANALYSIS	RANDOMIZED ALGORITHM
What is random	the **input** (you assume a distribution)	the **algorithm** (it calls a random-number generator)
Reported as	**average-case** running time	**expected** running time
Requires	knowing/assuming the input distribution	nothing about the input
Guarantee shape	"on average over inputs drawn from `D`…"	"on **any** input, in expectation over my coin flips…"
Bad case	a specific bad input exists	no bad input exists — only bad luck

> *In general, we discuss the **average-case running time** when the probability distribution is over the inputs to the algorithm, and we discuss the **expected running time** when the algorithm itself makes random choices. [CLRS §5.1, p.129]*

> *We call an algorithm **randomized** if its behavior is determined not only by its input but also by values produced by a random-number generator.*

The `RANDOM(a, b)` primitive. Returns an integer in `[a, b]` uniformly, independent of previous calls. *"You may imagine RANDOM as rolling a `(b − a + 1)`-sided die."* In practice you get a **pseudorandom** generator — a deterministic algorithm returning numbers that "look" statistically random.

Why the switch matters — CLRS's worked comparison

For the hiring problem (below), consider three inputs, listed as rank sequences:

INPUT	HIRES	CHARACTER
$A_1 = \langle 1,2,3,4,5,6,7,8,9,10 \rangle$	10	expensive
$A_2 = \langle 10,9,8,7,6,5,4,3,2,1 \rangle$	1	inexpensive
$A_3 = \langle 5,2,1,8,4,7,10,9,3,6 \rangle$	3	moderate

The deterministic algorithm's cost is a **fixed function of the input**. The randomized version first shuffles, so:

> *Given a particular input, say A_3, **we cannot say how many times the maximum is updated**, because this quantity differs with each run. … For this algorithm and many other randomized algorithms, **no particular input elicits its worst-case behavior. Even your worst enemy cannot produce a bad input array**, since the random permutation makes the input order irrelevant. The randomized algorithm performs badly only if the random-number generator produces an "unlucky" permutation. [CLRS §5.3, p.135]*

Skiena says the same thing about quicksort [§4.6.2, p.133]:

> *For any deterministic method of pivot selection, there exists a worst-case input instance which will doom us to quadratic time. … But now suppose we add an initial step where we randomly permute the order of the n elements. … **The worst case performance still can happen, but it now depends only upon how unlucky we are. There is no longer a well-defined "worst-case" input.***

The claim upgrades from
"Quicksort runs in $\theta(n \log n)$, with high probability, if you give it randomly ordered data"
to
*"Randomized quicksort runs in $\theta(n \log n)$ on **any** input, with high probability."*

Recognition pattern

You have an algorithm with a bad worst case, a good average case, and a *structured* input distribution in production (already-sorted data, adversarial keys, clustered values). Randomize.

---

2. Algorithm: The Hiring Problem

Problem [CLRS §5.1, p.126]

An agency sends one candidate per day. Interviewing costs c_i (cheap); hiring costs c_h (expensive — you must fire the incumbent and pay a fee). You always keep the best person seen so far. **How much do you expect to pay in hiring fees?**

```
HIRE-ASSISTANT(n)
1  best = 0                    // dummy candidate, less qualified
than everyone
2  for i = 1 to n
3      interview candidate i
4      if candidate i is better than candidate best
5          best = i
6          hire candidate i
```

→ **C++ implementation:** A1 HIRE-ASSISTANT

Why it matters — this is not really about hiring

> *This scenario serves as a model for a common computational paradigm. Algorithms often need to find the maximum or minimum value in a sequence by examining each element and maintaining a current "winner."* ***The hiring problem models how often a procedure updates its notion of which element is currently winning.***

So the answer applies to: how often does a running-max variable get written? how many times does a randomized incremental construction rebuild? how many prefix-maxima does a random permutation have?

Cost model

Total cost $O(c_i n + c_h \cdot m)$ where m = number hired. You always interview all n, so $c_i n$ is fixed; **all the analysis is about m**.

> *The analytical techniques used are identical whether we are analyzing cost or running time. In either case, we are counting the number of times certain basic operations are executed.*

Worst case

Candidates arrive in strictly increasing order of quality → you hire all n → $O(c_h \cdot n)$.

The probabilistic assumption

Candidates have a **total order**; `rank(i)` is candidate `i`'s rank (higher = better). "Random order" means `⟨rank(1), …, rank(n)⟩` is a **uniform random permutation** — each of the `n!` permutations has probability `1/n!`.

Analysis with indicator random variables

Define $X_i = I\{\text{candidate } i \text{ is hired}\}$, so $X = X_1 + X_2 + \cdots + X_n$.

The key probability. Candidate `i` is hired exactly when they are better than candidates `1..i−1`. Since the first `i` candidates appear in random order, **any one of them is equally likely to be the best so far**, so:

$$E[X_i] = \Pr\{\text{candidate } i \text{ is hired}\} = 1/i$$

By linearity of expectation:

$$E[X] = E[\Sigma\, X_i] = \Sigma\, E[X_i] = \Sigma_{i=1}^{n}\, 1/i = H_n = \ln n + O(1)$$

> *Even though you interview `n` people, you actually hire only approximately `ln n` of them, on average.*

Lemma 5.2. Average-case total hiring cost is `O(c_h ln n)` — a huge improvement over `O(c_h n)`.

The randomized version

```
RANDOMIZED-HIRE-ASSISTANT(n)
1  randomly permute the list of candidates
2  HIRE-ASSISTANT(n)
```

→ **C++ implementation:** A2 RANDOMIZED-HIRE-ASSISTANT and A3 RANDOMLY-PERMUTE

Lemma 5.3. Expected hiring cost is `O(c_h ln n)` — **with no assumption about the input**.

> *Lemma 5.2 makes an assumption about the input. Lemma 5.3 makes no such assumption, although randomizing the input takes some additional time.*

C++ Implementation

```cpp
#include <vector>
#include <random>
#include <cstdint>

// Counts how many times the running maximum is updated (== number
of "hires").
// Expected value on a random permutation: H_n = ln n + O(1).
int countHires(const vector<int>& rank) {
    int best = -1, hires = 0;
    for (int r : rank)
        if (r > best) { best = r; ++hires; }
    return hires;
}
```

Recognition pattern

"How many times does the running best get updated?", "how many prefix maxima / records does a random sequence have?", "how many times does a randomized incremental algorithm rebuild?" — all `Θ(log n)`, all by this argument.

---

3. Indicator Random Variables — the core technique

Definition [CLRS §5.2, p.130]

Given a sample space `S` and an event `A`:

```
I{A} = 1  if A occurs
       0  otherwise
```

Lemma 5.1 — the bridge between probability and expectation

> *Let `X_A = I{A}`. Then `E[X_A] = Pr{A}`.*

Proof. `E[X_A] = 1·Pr{A} + 0·Pr{Ā} = Pr{A}`. ∎

That is all it says, and it is the entire trick. It converts "what is the probability of…" into "what is the expected value of…", which lets you *add things up*.

Linearity of expectation — the reason this works

$$E[X_1 + X_2 + \cdots + X_n] = E[X_1] + E[X_2] + \cdots + E[X_n]$$

> ***Linearity of expectation applies even when there is dependence among the random variables.*** *[CLRS §5.2, p.131]*

This is the single most useful fact in randomized analysis. You almost never have to check independence to add expectations. (Independence *is* needed to multiply probabilities — a different operation, and the place people conflate the two.)

The recipe

1. Identify the quantity `X` you want the expectation of — it must be a **count** of something.
2. Break it into indicators: `X = Σ Xᵢ` where `Xᵢ` indicates "the `i`-th thing happened".
3. Compute `E[Xᵢ] = Pr{i-th thing happens}` — usually easy, often by symmetry.
4. Sum. Don't check independence.

Warm-up: expected heads in `n` coin flips

Direct approach: consider `Pr{0 heads}, Pr{1 head}, …` separately and evaluate a binomial sum. Painful.

Indicator approach: `Xᵢ = I{flip i is heads}`, `E[Xᵢ] = 1/2`, `X = ΣXᵢ`, so `E[X] = n/2`. Two lines.

> *Compared with the [binomial-sum] method, indicator random variables* ***greatly simplify the calculation.***

Worked exercises — do these, they are the whole skill

PROBLEM	SETUP	ANSWER
Sum of `n` dice [Ex. 5.2-3]	`Xᵢ` = value of die `i`; `E[Xᵢ] = 3.5`	`3.5n`
Hat-check [Ex. 5.2-5]	`n` hats returned in random order; `Xᵢ = I{customer i gets own hat}`, `E[Xᵢ] = 1/n`	`n·(1/n) = 1`
Inversions [Ex. 5.2-6]	`X_{ij} = I{A[i] > A[j]}` for `i < j`; by symmetry `E = 1/2`	`C(n,2)/2 = n(n−1)/4`
Dependent dice [Ex. 5.2-4]	die 2 set equal to die 1; still `E[sum] = 7`	linearity holds under full dependence

The hat-check answer — **exactly 1, for every `n`** — is the one to remember, because it is so obviously not derivable by any direct counting argument.

The inversions answer is also the average-case running time of insertion sort M01: `Θ(n + n²/4) = Θ(n²)`.

---

4. Algorithm: Fisher–Yates (Randomly Permuting an Array)

Problem

Produce a **uniform random permutation** of `A[1..n]` — every one of the `n!` permutations with probability exactly `1/n!`.

The trap up front

> *You might think that to prove that a permutation is a uniform random permutation, it suffices to show that, for each element `A[i]`, the probability that the element winds up in position `j` is `1/n`. **This weaker condition is, in fact, insufficient.** [CLRS §5.3, p.136]*

The counterexample is `PERMUTE-BY-CYCLE` [Ex. 5.3-4]: pick one random `offset` and rotate. Every element does land in every position with probability `1/n` — but only `n` of the `n!` permutations are ever produced.

Algorithm

```
RANDOMLY-PERMUTE(A, n)
1  for i = 1 to n
2      swap A[i] with A[RANDOM(i, n)]        ← note the range:
i..n, NOT 1..n
```

→ **C++ implementation:** A2 RANDOMIZED-HIRE-ASSISTANT and A3 RANDOMLY-PERMUTE

In place, `θ(n)` time, `O(1)` extra space. After iteration `i`, `A[i]` is never touched again.

Proof skeleton — loop invariant on `k`-permutations

A **`k`-permutation** is a sequence of `k` of the `n` elements, no repetitions. There are `n!/(n−k)!` of them.

> ***Invariant:*** *Just prior to the `i`-th iteration, for each possible `(i−1)`-permutation of the `n` elements, the subarray `A[1..i−1]` contains that `(i−1)`-permutation with probability `(n−i+1)!/n!`.*

- **Initialization** (`i = 1`). `A[1..0]` is empty and a 0-permutation has no elements, so it contains "the" 0-permutation with probability 1, matching `(n−1+1)!/n! = n!/n! = 1`. ✓
- **Maintenance.** Fix a target `i`-permutation `⟨x₁,…,xᵢ⟩`. Let `E₁` = "the first `i−1` iterations produced `⟨x₁,…,x_{i−1}⟩`", so `Pr{E₁} = (n−i+1)!/n!` by the invariant. Let `E₂` = "iteration `i` puts `xᵢ` in `A[i]`". Line 2 chooses uniformly from the `n−i+1` values in `A[i..n]`, so `Pr{E₂ | E₁} = 1/(n−i+1)`. Hence

  ```
  Pr{E₂ ∩ E₁} = Pr{E₂ | E₁}·Pr{E₁} = 1/(n−i+1) · (n−i+1)!/n! =
  (n−i)!/n!    ✓
  ```

- **Termination.** `i = n+1`, so `A[1..n]` is any given `n`-permutation with probability `(n−(n+1)+1)!/n! = 0!/n! = 1/n!`. ∎

C++ Implementation

```
#include <vector>
#include <random>
#include <utility>
```

```cpp
// Fisher-Yates (Durstenfeld). Uniform over all n! permutations.
Theta(n), in place.
template <typename T, typename RNG>
void shuffleUniform(vector<T>& a, RNG& rng) {
    const int n = static_cast<int>(a.size());
    for (int i = 0; i < n - 1; ++i) {
        // CRITICAL: draw from [i, n-1], not [0, n-1].
        uniform_int_distribution<int> pick(i, n - 1);
        swap(a[i], a[pick(rng)]);
    }
}

// In practice, prefer the standard library:
//   mt19937 rng(random_device{}());
//   shuffle(a.begin(), a.end(), rng);
// random_shuffle was removed in C++17 -- do not use it.
```

Common bugs — three wrong variants, all from CLRS's exercises

VARIANT	WHAT IT DOES	VERDICT
`swap A[i] with A[RANDOM(1, n)]` (`PERMUTE-WITH-ALL`) [Ex. 5.3-3]	picks from the **whole** array each time	**Wrong.** Produces n^n equally likely execution paths; n^n is not divisible by $n!$ for $n > 2$, so the permutations cannot be equiprobable.
`for i = 1 to n-1: swap A[i] with A[RANDOM(i+1, n)]` (`PERMUTE-WITHOUT-IDENTITY`) [Ex. 5.3-2]	tries to exclude the identity	**Wrong.** Excludes far more than the identity — e.g. it can never leave `A[1]` in place at all.
`PERMUTE-BY-CYCLE` [Ex. 5.3-4]	one random rotation	**Wrong.** Only `n` of `n!` permutations reachable, though each element hits each position with probability `1/n`.

Also: using `rand() % n` for the index introduces **modulo bias** unless `n` divides `RAND_MAX + 1`. Use `std::uniform_int_distribution`.

Related: sampling `m` of `n` with only `m` random draws

When `n ≫ m`, shuffling all of `A` wastes `n` calls to `RANDOM`. [CLRS Ex. 5.3-5]:

```
RANDOM-SAMPLE(m, n)
1  S = ∅
2  for k = n − m + 1 to n            // iterates m times
3      i = RANDOM(1, k)
4      if i ∈ S:  S = S ∪ {k}
5      else:      S = S ∪ {i}
6  return S
```

→ **C++ implementation:** A4 RANDOM-SAMPLE

Every `m`-subset is equally likely, using only `m` calls to `RANDOM`.

Recognition pattern

Any time you need to destroy adversarial input structure: randomized quicksort, randomized incremental geometry, shuffling training data, A/B bucket assignment, load balancing.

---

5. Las Vegas vs Monte Carlo

The taxonomy

	LAS VEGAS	MONTE CARLO
Correctness	**always correct**	correct **with high probability**
Running time	random (**expected** bound)	**deterministic** / bounded
Guarantee traded away	time	correctness
Examples	randomized quicksort, Fisher–Yates, randomized selection, treaps	Fermat/Miller–Rabin primality, Karger's min cut, Monte Carlo integration, Bloom filters

Skiena's compressed version [§6.8, p.191]:

> ***Monte Carlo algorithms are always fast, usually correct, and most of them are wrong in only one direction.***

That last clause is the practical one. **One-sided error** is far more useful than two-sided: if a Bloom filter says "not present", it is *certainly* not present; if Miller–Rabin says "composite", it is *certainly* composite. You can design around a one-sided error; you can't design around a coin flip.

Converting between them. A Las Vegas algorithm with expected time `T` becomes Monte Carlo by cutting it off at `kT` and reporting failure (Markov's inequality bounds the failure probability by `1/k`). Going the other way requires a way to *verify* an answer.

Skiena's four design patterns for randomized algorithms [§4.6.2, p.134]

PATTERN	IDEA	WHERE
Random sampling	Want the median of `n` things but can't touch them all? Take a small random sample and use its median. *"This is the idea behind opinion polling… Biases creep in unless you take a truly random sample, as opposed to the first `x` people you happen to see."*	quickselect pivots, approximate quantiles, reservoir sampling
Randomized hashing	For any fixed hash function there is a worst-case key set. Pick the function at random from a large family.	universal hashing, M07
Randomized search	Drive a search with randomness.	simulated annealing, M20 *(planned)*
Random shuffle / random pivot	Destroy input structure before running a structure-sensitive algorithm.	quicksort, M05

Stop and Think: Nuts and Bolts [Skiena §4.6.2, p.134]

Problem. `n` bolts of different widths and `n` matching nuts. You may test a nut against a bolt (too big / too small / exact), but **you cannot compare two nuts or two bolts directly**. Match them all.

`O(n²)`: for each bolt, scan the nuts.

Randomized $O(n \log n)$ expected: emulate quicksort. Pick a random bolt b. Partition the *nuts* into those smaller and larger than b — and in doing so you find b's matching nut ν. Now use ν to partition the *bolts*. That is $2n - 2$ comparisons for a partition step; the rest of the analysis is exactly randomized quicksort's.

> *What is interesting about this problem is that* ***no simple deterministic algorithm for nut and bolt sorting is known.*** *It illustrates how randomization makes the bad case go away, leaving behind a simple and beautiful algorithm.*

(A deterministic $\Theta(n \log n)$ algorithm does exist — Komlós, Ma, Szemerédi — but it is complicated.)

---

6. Four canonical probabilistic analyses

These four recur constantly. Learn the setup, not just the answer.

6.1 The Birthday Paradox

Question. How many people before a 50% chance that two share a birthday, with $n = 365$ days?

Exact analysis via complements. Let B_k = "all k birthdays distinct".

$$
\begin{aligned}
\Pr\{B_k\} &= 1 \cdot (1 - 1/n)(1 - 2/n) \cdots (1 - (k-1)/n) \\
&\le e^{-1/n} \cdot e^{-2/n} \cdots e^{-(k-1)/n} && \text{[using } 1 + x \le e^x\text{]} \\
&= e^{-k(k-1)/2n} \\
&\le 1/2 && \text{when } k(k-1) \ge 2n \ln 2
\end{aligned}
$$

Solving the quadratic: $k \ge (1 + \sqrt{(1 + (8 \ln 2)n)})/2$. For $n = 365$ → $\mathbf{k \ge 23}$.

*(CLRS's aside: a Martian year is 669 days, so it takes **31 Martians**.)*

Approximate analysis via indicators — much shorter. $X_{ij} = I\{i \text{ and } j \text{ share a birthday}\}$, $E[X_{ij}] = 1/n$.

$$
E[X] = C(k,2) \cdot (1/n) = k(k-1)/(2n)
$$

Expected number of matching pairs reaches 1 when `k(k−1) ≥ 2n`, i.e. `k ≈ √(2n) + 1`. For `n = 365`, `k = 28` gives `E[X] ≈ 1.0356`.

The two answers differ (23 vs 28) because they ask different questions — "probability exceeds ½" vs "expected count reaches 1" — but both are **`Θ(√n)`**.

Why it matters algorithmically: collisions appear after `Θ(√n)` insertions into `n` slots. This is why hash tables collide far earlier than intuition suggests, why 64-bit hashes start colliding around 2^{32}, and why birthday attacks halve the effective bit-security of a hash function.

6.2 Balls and Bins

Toss `n` balls independently and uniformly into `n` bins. [CLRS §5.4.2; Skiena §6.2]

QUESTION	ANSWER
Balls in a given bin (`n` balls, `b` bins)	binomial `b(k; n, 1/b)`, mean **`n/b`**
Tosses until a given bin has a ball	geometric, mean **`b`**
Tosses until **every** bin has a ball	**`b(ln b + O(1)) ≈ b ln b`** — coupon collector
Fraction of bins **empty** (`n` balls, `n` bins)	**`1/e ≈ 36.79%`**
Fraction of bins with **exactly one** ball	also **`1/e`**
Maximum load	**`Θ(log n / log log n)`**

Skiena's simulation [§6.2, p.179] — the numbers are worth seeing because they are so stable:

ITEMS k IN BUCKET	$N = 10^6$	$N = 10^7$	$N = 10^8$
0	367,899	3,678,774	36,789,634
1	367,928	3,677,993	36,785,705
2	183,926	1,840,437	18,392,948
3	61,112	613,564	6,133,955
4	15,438	152,713	1,531,360
5	3,130	30,517	306,819
6	499	5,133	51,238
7	56	754	7,269
8	12	107	972
9	—	8	89
10	—	—	10
11	—	—	1

> ***36.78% of the buckets are empty in all three cases. That can't be a coincidence.***

Why `1/e`. Bucket 1 is empty iff all `n` balls land elsewhere:

$$\Pr\{|B_1| = 0\} = ((n-1)/n)^n \;\to\; 1/e = 0.367879\ldots$$

And the honest correction Skiena makes:

> *The fullest bucket gets fuller as `n` increases, from 8 to 9 to 11. In fact, the expected value of the longest list is `O(log n / log log n)`, which grows slowly but is **not a constant**. Thus, I was a little too glib when I said in §3.7.1 that the worst-case access time for hashing is `O(1)`.*
>
> *(footnote) To be precise, the expected search time for hashing is `O(1)` **averaged over all `n` keys**, but we also expect there will be a few keys unlucky enough to require `Θ(log n / log log n)` time.*

Engineering consequence: hash-table *tail latency* is not `O(1)`. If your p99.9 matters, this is why. (The fix — "the power of two choices", picking the emptier of two random bins, which drops max load to `Θ(log log n)` — is outside both books.)

6.3 Coupon Collector

Question. Keep tossing balls into `b` bins until none is empty. How many tosses?

Setup. Split the tosses into **stages**: stage `i` runs from just after the `(i−1)`-st *hit* (a ball landing in an empty bin) through the `i`-th hit. During stage `i`, exactly `i−1` bins are full, so `Pr{hit} = (b − i + 1)/b`. The number of tosses in a stage is **geometric**, so `E[nᵢ] = b/(b − i + 1)`.

```
E[n] = Σ_{i=1}^{b} b/(b−i+1) = b·Σ_{i=1}^{b} 1/i = b·H_b = b(ln b + O(1)) ≈ b ln b
```

The reindexing `Σ 1/(b−i+1) = Σ 1/i` is CLRS eq. (A.14) — the trick from M02.

> *If you are trying to collect each of `b` different coupons, then you should expect to acquire approximately **`b ln b`** randomly obtained coupons in order to succeed.*

Where it shows up: cache warm-up, testing all code paths with random inputs, random-walk covering time, distributed systems waiting for every shard to report.

Random-walk covering times [Skiena §6.1.6, §6.2.1 Stop-and-Think]

Two graphs, two wildly different answers:

GRAPH	EXPECTED COVERING TIME	ARGUMENT
Path on `m` vertices	**`Θ(m²)`**	You need `m−1` more heads than tails. The head–tail difference has spread `σ = Θ(√n)` after `n` flips, so you need `m = Θ(√n)`, i.e. `n = Θ(m²)`.
Complete graph K_n	**`Θ(n log n)`**	Each step is essentially a uniform random draw → coupon collector. (Self-loops are disallowed, changing `(n−i)/n` to `(n−i)/(n−1)` and `nH_n` to `(n−1)H_n` — asymptotically identical.)

6.4 Streaks

Question. Longest run of consecutive heads in `n` fair flips? **Answer: `Θ(lg n)`.** [CLRS §5.4.3]

Upper bound $O(\lg n)$. $\Pr\{\text{a specific position starts a run of} \geq k\} = 1/2^k$. At $k = 2\lceil \lg n \rceil$ this is $\leq 1/n^2$. Boole's inequality over $\leq n$ starting positions gives $\Pr\{\text{any run of length} \geq 2\lceil \lg n \rceil\} < 1/n$. Split $E[L] = \Sigma\, j \cdot \Pr\{L_j\}$ at $j = 2\lceil \lg n \rceil$: below it j is small, above it the probability is small.

$$E[L] < 2\lceil \lg n \rceil \cdot 1 + n \cdot (1/n) = O(\lg n)$$

Lower bound $\Omega(\lg n)$. Partition into $\lfloor n/\lfloor (\lg n)/2 \rfloor \rfloor$ disjoint groups of $s = \lfloor (\lg n)/2 \rfloor$ flips. $\Pr\{\text{a group is all heads}\} \geq 1/\sqrt{n}$. Groups are independent, so $\Pr\{\text{no group all heads}\} = O(1/n)$, giving $E[L] \geq s(1 - O(1/n)) = \Omega(\lg n)$.

Tail bound worth memorizing: $\Pr\{\text{some run of length} \geq r\lceil \lg n \rceil\} \leq 1/n^{r-1}$. So in $n = 1000$ flips, a run of 20 heads has probability $\leq 1/1000$; a run of 30 has probability $\leq 1/10^6$.

Indicator shortcut. $E[X_k]$ = expected number of runs of length $\geq k$ = $(n - k + 1)/2^k$. At $k = c \lg n$ this is $\Theta(1/n^{c-1})$ — large when $c < 1$, tiny when $c > 1$, pinning the answer at $\Theta(\lg n)$.

Why it matters: longest probe sequence in linear-probing hash tables, longest run in skip lists, and the intuition behind "log-depth with high probability" arguments generally.

6.5 The Secretary Problem (online hiring) — the $1/e$ rule

[CLRS §5.4.4, p.150]

Problem. Same candidates, but you must accept or reject **immediately**, and you hire exactly once. Maximize the probability of getting the best candidate.

Strategy. Reject the first k, then hire the first subsequent candidate better than all of the first k. (If the best was in the first k, you end up with candidate n.)

```
ONLINE-MAXIMUM(k, n)
1  best-score = −∞
2  for i = 1 to k
3      if score(i) > best-score: best-score = score(i)
4  for i = k + 1 to n
5      if score(i) > best-score: return i
6  return n
```

→ **C++ implementation:** A5 ONLINE-MAXIMUM (the secretary problem)

Analysis. Let S_i = "you succeed and the best is at position i". Success needs two independent events: B_i = "the best is at position i" ($Pr = 1/n$), and O_i = "nobody in positions $k+1..i-1$ was picked", which happens iff the max of positions $1..i-1$ lies in the first k ($Pr = k/(i-1)$). These are independent — O_i depends only on the *relative ordering* of the first $i-1$, while B_i depends only on whether position i beats everything.

$$Pr\{S\} = \Sigma_{i=k+1}^{n} (1/n)(k/(i-1)) = (k/n)\cdot\Sigma_{i=k}^{n-1} 1/i$$

Bounding the sum by integrals: $(k/n)(\ln n - \ln k) \le Pr\{S\} \le (k/n)(\ln(n-1) - \ln(k-1))$.

Maximize the lower bound: $d/dk[(k/n)(\ln n - \ln k)] = (1/n)(\ln n - \ln k - 1) = 0 \implies \ln k = \ln(n/e) \implies \mathbf{k = n/e}$.

Result: observe $n/e \approx 37\%$ of candidates, then take the first record-breaker. Succeeds with probability at least $1/e \approx 37\%$.

This is one of the most-cited results in online algorithms — see M24 *(planned)* for the competitive-analysis framing.

---

7. Why hashing needs randomness

[Skiena §6.3, p.181 — this argument has no counterpart in CLRS Ch. 5 and is worth its own section]

The problem

A hash function must be **deterministic** — $h(x)$ has to give the same answer every time or you could never find x again. So where does randomness enter?

> *One reason we like randomized algorithms is that they make the worst-case input instance go away: bad performance should be a result of **extremely bad luck**, rather than some joker giving us data that makes us do bad things. But it is easy (in principle) to construct a worst case example for **any** hash function h.*

The pigeonhole argument. Take any set `S` of `nm` distinct keys and hash them all. The range has only `m` slots, so the average bucket holds `nm/m = n` items, and by pigeonhole **some bucket has ⩾ `n` items**. Those `n` keys, presented alone, are a worst case for `h`.

The fix

> *We are protected if we **pick our hash function at random from a large set of possibilities**, because we can only construct such a bad example by knowing the exact hash function we will be working with.*

Constructing a random family

Typically `h(x) = f(x) mod m`, where `f` maps the key to a huge value and `m` is fixed by memory constraints — so you can't randomize `m`. Instead, insert a randomly chosen prime `p` in the middle. Note that in general

```
f(x) mod m  ≠  (f(x) mod p) mod m
```

(Skiena's example: `21347895537127 mod 17 = 8`, but `(21347895537127 mod 2342343) mod 17 = 12`.) So define

```
h(x) = ((f(x) mod p) mod m)
```

with `p` chosen at random, and it works out **provided** (a) `f(x)` is large relative to `p`, (b) `p` is large relative to `m`, (c) `m` is relatively prime to `p`.

> *This ability to select random hash functions means we can now use hashing to provide **legitimate randomized guarantees**, thus making the worst-case input go away.*

Full treatment of universal and perfect hashing in M07.

Engineering note

This is not academic. **Hash-flooding denial-of-service attacks** exploit exactly this: an attacker who knows your language runtime's hash function crafts colliding keys and turns an `O(1)` dictionary into `O(n)`. The mitigation deployed across Python, Ruby, PHP, Node and the Linux kernel is **hash randomization with a per-process random seed** — Skiena's argument, in production.

---

8. Algorithm: Randomized Primality Testing

Problem

Decide whether `n` is prime.

Why trial division fails

Loop `i` from 2 to `⌊√n⌋`. That is `O(√n)` — **in the value of `n`, not its bit length**. A 1024-bit RSA key needs `√(2^1024) = 2^512` divisions, *"which is greater than the number of atoms in the universe."* [Skiena §6.8, p.190]

Note the input-size subtlety (from M02): for number-theoretic problems, `n` is the *value* but input size is `lg n` bits, so `O(√n)` is **exponential** in the input size.

Core intuition — Fermat's little theorem

> *If `n` is prime, then `a^{n−1} ≡ 1 (mod n)` for all `a` not divisible by `n`.*

Examples: `n = 17, a = 3` → `3¹⁶ ≡ 1 (mod 17)` ✓. `n = 16, a = 3` → `3¹⁵ ≡ 11 (mod 16)` ✗, proving 16 composite.

> *What makes this interesting is that the mod of this big power **always** is 1 if `n` is prime. This is a pretty good trick, because the odds of it being 1 by chance should be very small — only `1/n` if the residue was uniform in the range.*

The algorithm

Pick 100 random `a` in `[1, n−1]`. Verify none divides `n`. Compute `a^{n−1} mod n`. If all hundred give 1, the probability `n` is composite is `< (1/2)¹⁰⁰` — vanishingly small.

> *Because the number of tests (100) is fixed, **the running time is always fast**, which makes this a **Monte Carlo** type of randomized algorithm.*

The Carmichael caveat

> *A very small fraction of integers (roughly 1 in 50 billion up to `10²¹`) are not prime, yet also satisfy the Fermat congruence **for all `a`**. Such **Carmichael numbers** like 561 and 1105 are doomed to always be misclassified as prime.*

This is why you use Miller–Rabin, not plain Fermat. Miller–Rabin adds a check on non-trivial square roots of 1 and defeats Carmichael numbers, with error `≤ 4^{-k}` for `k` rounds. Full treatment in M21 *(planned)*.

Complexity

`a^{n-1} mod n` takes `O(log n)` multiplications by binary exponentiation (M02 §8). And crucially:

```
(x · y) mod n = ((x mod n) · (y mod n)) mod n
```

> *…so we **never need multiply numbers larger than** `n` over the course of the computation.*

Total: `O(k log³ n)` bit operations for `k` rounds using schoolbook multiplication.

C++ Implementation

```
#include <cstdint>
#include <random>
#include <vector>

static uint64_t mulMod(uint64_t a, uint64_t b, uint64_t m) {
    return static_cast<uint64_t>((static_cast<__uint128_t>(a) * b)
% m);
}

static uint64_t powMod(uint64_t b, uint64_t e, uint64_t m) {
    uint64_t r = 1 % m;
    b %= m;
    while (e) { if (e & 1) r = mulMod(r, b, m); b = mulMod(b, b,
m); e >>= 1; }
    return r;
}

// Deterministic Miller-Rabin for all 64-bit n, using the known
witness set.
// (Randomized MR would draw `a` uniformly from [2, n-2] instead.)
bool isPrime(uint64_t n) {
    if (n < 2) return false;
```

```
    for (uint64_t p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                       19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (n % p == 0) return n == p;
    }
    // Write n - 1 = d * 2^s with d odd.
    uint64_t d = n - 1;
    int s = 0;
    while ((d & 1) == 0) { d >>= 1; ++s; }

    for (uint64_t a : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                       19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        uint64_t x = powMod(a, d, n);
        if (x == 1 || x == n - 1) continue;          // a is not a
witness
        bool composite = true;
        for (int i = 1; i < s; ++i) {                // repeatedly
square
            x = mulMod(x, x, n);
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;                 // a witnesses
compositeness
    }
    return true;
}
```

Implementation notes

- The **squaring loop** is exactly what plain Fermat lacks: it detects a non-trivial square root of 1 mod `n`, which cannot exist if `n` is prime. This is what kills Carmichael numbers.
- The 12 fixed bases `{2,3,5,…,37}` make the test **deterministic and correct for every `n < 2⁶⁴`** — a known result. For larger `n`, draw `a` at random and it becomes genuinely Monte Carlo.
- `__int128` for `mulMod`; without it you need Montgomery multiplication or a modulus below `2³¹`.
- Small-prime trial division first is a large constant-factor win on random inputs.

Recognition pattern

Any problem where **verifying a random certificate is cheap and a failed verification is conclusive**. That is the general shape of a one-sided-error Monte Carlo algorithm: polynomial identity testing (Schwartz–Zippel), matrix product verification (Freivalds), primality.

---

9. Where do random numbers come from?

[Skiena §6.10, p.192]

We picture coin flips and Geiger counters. **That is not what your RNG does.** Most library generators are essentially a hash of the previous output — a **linear congruential generator**:

$$R_n = (a \cdot R_{n-1} + c) \bmod m$$

with a, c, m, R_0 large, carefully chosen constants.

> *The alert reader may question exactly how random such numbers really are. Indeed, they are **completely predictable**, because knowing R_{n-1} provides enough information to construct R_n. This predictability means that a sufficiently determined adversary could in principle **construct a worst-case input to a randomized algorithm** provided they know the current state of your random number generator.*

> *Linear congruential generators are more accurately called **pseudo-random** number generators. The stream looks random, in that it has the same statistical properties as a truly random source. This is generally good enough for randomized algorithms to work well in practice. However, there is a philosophical sense of randomness which has been lost that occasionally comes back to bite us, typically in **cryptographic applications** whose security guarantees rest on an assumption of true randomness.*

Outside / Engineering Context

Neither book covers current practice, so, clearly flagged as outside knowledge: in modern C++ use `<random>` — `std::mt19937` (or `mt19937_64`) seeded from `std::random_device`, with `std::uniform_int_distribution` rather than `%`. Mersenne Twister is *not* cryptographically secure (its state is recoverable from 624 consecutive outputs); for anything security-relevant use the OS CSPRNG (`getrandom(2)`, `/dev/urandom`, `BCryptGenRandom`). `std::rand()` and the removed `std::random_shuffle` should not be used. For competitive programming, seed from `std::chrono::steady_clock` so that hack-resistant behaviour doesn't depend on a fixed seed.

10. What "expected time" actually feels like

Skiena's most practically useful paragraph in the chapter [§6.1.6, p.178]:

> ***Take-Home Lesson:*** *Students often ask me "what happens" when randomized quicksort runs in* $\theta(n^2)$*. The answer is that* ***nothing happens****, in exactly the same way nothing happens when you buy a lottery ticket: you almost certainly just lose. With a randomized quicksort* ***you almost certainly just win****: the probability distribution is so tight that you nearly always run in time very close to expectation.*

The mathematical backing. The binomial distribution for h heads in $n = 10{,}000$ fair flips has $\sigma = \sqrt{(np(1-p))} = \theta(\sqrt{n})$ — for $n = 10^4$, $\sigma = 50$ against a mean of 5,000. **Chebyshev's inequality**: for *any* distribution, at least $1 - 1/k^2$ of the mass lies within $\pm k\sigma$ of the mean.

> *Typically* σ *is small relative to* μ *for the distributions arising in the analysis of randomized algorithms.*

Why this matters for engineering judgment: an expected-$O(n \log n)$ randomized algorithm is not a gamble in the way people fear. The $\theta(n^2)$ outcome for randomized quicksort on $n = 10^6$ has probability so small it is dominated by the chance of a cosmic-ray bit flip. Design accordingly — and be equally aware that *this reasoning does not apply to hash-table tail latency*, where the max-load bound really is $\theta(\log n / \log \log n)$ and really does show up in your p99.9.

Chapter in One Page

CONCEPT	THE ONE-LINE VERSION
Probabilistic analysis	Randomness in the **input**; report **average-case** time; needs an assumed distribution.
Randomized algorithm	Randomness in the **algorithm**; report **expected** time; works on any input.
The core payoff	Randomization makes the **worst-case input** go away — only bad luck remains.
Indicator RV	$I\{A\} = 1$ if A occurs, else 0. $\mathbf{E[I\{A\}] = Pr\{A\}}$ (Lemma 5.1).
Linearity of expectation	$E[\Sigma X_i] = \Sigma E[X_i]$ — **holds even under dependence**.

The recipe	Count → decompose into indicators → `E[Xᵢ] = Pr{…}` → sum. Never check independence.
Hiring problem	`E[hires] = H_n = ln n + O(1)`. Models "how often does the running max update".
Hat-check	Expected number getting their own hat is **exactly 1**, for every `n`.
Inversions	`E = n(n−1)/4` — the average case of insertion sort.
Fisher–Yates	`swap A[i] with A[RANDOM(i, n)]` — range `i..n`, **not** `1..n`. `Θ(n)`, uniform over `n!`.
The `1/n` trap	Every element landing in every position with probability `1/n` does **not** imply uniformity.
Las Vegas	Always correct, expected-time bound. (quicksort, shuffling, treaps)
Monte Carlo	Always fast, usually correct — **most are wrong in only one direction**. (Miller–Rabin, Bloom)
Birthday paradox	Collisions at `Θ(√n)`. 23 people for 50%; 28 for expected-1-pair.
Balls & bins (`n`,`n`)	`1/e ≈ 36.8%` empty, `1/e` singletons, **max load `Θ(log n / log log n)`**.
Hashing is not worst-case `O(1)`	Averaged over keys, yes. A few unlucky keys cost `Θ(log n / log log n)`.
Coupon collector	`b·H_b ≈ b ln b` tosses to fill every bin.
Covering time	Path: `Θ(m²)`. Complete graph: `Θ(n log n)`.
Streaks	Longest head-run in `n` flips is `Θ(lg n)`; `Pr{run ≥ r⌈lg n⌉} ≤ 1/n^{r−1}`.
Secretary problem	Skip `n/e` (≈37%), then take the first record. Wins with probability `≥ 1/e`.
Why hashing needs randomness	Pigeonhole gives a bad key set for **any** fixed `h`. Pick `h` at random from a family.
Random hash family	`h(x) = ((f(x) mod p) mod m)` with random prime `p`.
Fermat test	`a^{n−1} ≡ 1 (mod n)` if `n` prime. Fooled by **Carmichael numbers** (561, 1105).
Miller–Rabin	Fermat + non-trivial-square-root check. Error `≤ 4^{−k}`. `O(k log³n)` bit ops.
Modular exponentiation	`O(log n)` multiplies; intermediates never exceed `n`.
PRNGs	LCG `Rₙ = (aRₙ₋₁ + c) mod m` — deterministic and predictable.

	Fine for algorithms, **not for crypto**.
Concentration	`σ = Θ(√n)` vs `μ = Θ(n)`; Chebyshev gives `1 − 1/k²` within `±kσ`. Randomized quicksort essentially never loses.

Recognition Table

CLUE	TECHNIQUE
"Expected number of X" where X is a count	**indicator RVs + linearity of expectation**
The events are clearly dependent	linearity still applies — don't be scared off
`Σ 1/i` appears	harmonic → `Θ(log n)`
"Until every one of `b` things has been hit"	coupon collector → `b ln b`
"How many until the first success", probability `p`	geometric → `1/p`
"Probability that two of `k` collide among `n`"	birthday → threshold at `Θ(√n)`
"Longest run of consecutive successes"	streaks → `Θ(log n)`
Must decide online, irrevocably, maximize chance of picking the best	secretary → skip `n/e`
Algorithm has bad worst case, good average case	**shuffle the input, or randomize the pivot**
Adversary can craft bad inputs for your deterministic choice	pick that choice at random from a family (universal hashing)
Need to sample `m` of `n` with `n ≫ m`	`RANDOM-SAMPLE`, `m` draws — not a full shuffle
Verifying a candidate answer is cheap, failure is conclusive	one-sided Monte Carlo (primality, Freivalds, Schwartz–Zippel)
Need a bound on how far from the mean	Chebyshev (`1 − 1/k²`) or Chernoff for tighter
Hash-table p99 latency is bad	max load is `Θ(log n / log log n)`, not `O(1)`

Common Mistakes Recap

1. Saying "average-case" when you mean "expected", or vice versa. They quantify over different things.
2. Checking independence before applying **linearity of expectation** — it isn't needed.
3. Multiplying probabilities without checking independence — it **is** needed there.
4. Fisher–Yates with `RANDOM(1, n)` instead of `RANDOM(i, n)`.
5. Believing "each element lands in each position with probability `1/n`" proves uniformity.
6. `rand() % n` — modulo bias.
7. Claiming hashing is worst-case `O(1)` per operation. It is `O(1)` **averaged over keys**.
8. Using plain Fermat testing and being fooled by Carmichael numbers.
9. Using Mersenne Twister (or any LCG) where cryptographic randomness is required.
10. Treating the `θ(n²)` case of randomized quicksort as a practical risk — the distribution is far too tight.
11. Assuming `O(√n)` primality testing is efficient — it is exponential in the input's **bit length**.
12. Forgetting that a randomized algorithm's guarantee evaporates if the adversary can see or predict your RNG state.

Self-Test

1. Distinguish probabilistic analysis from a randomized algorithm, and average-case from expected running time. *(§1)*
2. Why can no adversary construct a worst-case input for randomized quicksort? *(§1)*
3. State Lemma 5.1 and prove it in one line. *(§3)*
4. Under what conditions does linearity of expectation hold? *(§3)*
5. Show `E[hires] = H_n` for the hiring problem. Why is `Pr{candidate i hired} = 1/i`? *(§2)*
6. Expected number of customers getting their own hat back, out of `n`? *(§3)*
7. Expected number of inversions in a random permutation, via indicators? *(§3)*
8. Write Fisher–Yates. Why `RANDOM(i, n)`? Give the loop invariant. *(§4)*
9. Give three wrong shuffle variants and say what's wrong with each. *(§4)*

10. Las Vegas vs Monte Carlo — which guarantee does each give up? Give two examples of each. *(§5)*
11. Solve nuts-and-bolts in `O(n log n)` expected. Why is randomization essential here? *(§5)*
12. Birthday paradox: give both the exact and the indicator analysis. Why do they give 23 and 28? *(§6.1)*
13. Toss `n` balls into `n` bins: what fraction of bins are empty, and why exactly `1/e`? *(§6.2)*
14. What is the expected maximum load, and what does that mean for hash-table latency? *(§6.2)*
15. Derive `b·H_b` for coupon collector by staging. *(§6.3)*
16. Covering time of a path vs a complete graph, with the argument for each. *(§6.2)*
17. What is the expected longest head-streak in `n` flips? Sketch both bounds. *(§6.4)*
18. Secretary problem: what is the optimal `k`, and how is it derived? *(§6.5)*
19. Give the pigeonhole argument for why any fixed hash function has a bad key set, and the fix. *(§7)*
20. State Fermat's little theorem and the primality algorithm built on it. What are Carmichael numbers? *(§8)*
21. Why is `O(√n)` trial division exponential? *(§8)*
22. Write Miller–Rabin. Which line defeats Carmichael numbers? *(§8)*
23. What is an LCG, and in what setting is it unsafe? *(§9)*
24. What actually happens when randomized quicksort "hits its worst case"? *(§10)*

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
`RANDOMLY-PERMUTE` (Fisher–Yates)	384 · Shuffle an Array	write it with `RANDOM(i, n)` and then check uniformity yourself — `A3` shows how
Reservoir sampling, `k = 1`	382 · Linked List Random Node · 398 · Random Pick Index	one pass, `O(1)` memory, unknown stream length: the online cousin of `RANDOM-SAMPLE`
Weighted sampling	528 · Random Pick with Weight	prefix sums + binary search — randomization meets M03
Randomized `O(1)` structure	380 · Insert Delete GetRandom O(1)	the swap-with-last trick; uniform random element from a dynamic set
Randomized selection	215 · Kth Largest Element in an Array	quickselect's `O(n)` **expected** time is this module's payoff — see M05
Randomized pivoting	912 · Sort an Array	submit a deterministic-pivot quicksort, watch it TLE on the adversarial test, then randomize
Hashing with a random seed	1044 · Longest Duplicate Substring	Rabin–Karp with a randomly chosen base is the intended solution — see M07

Beyond LeetCode. Codeforces `probabilities` tag · `randomized` tag. CSES Problem Set — *Mathematics*.

The drill that matters here: whenever you write a randomized routine, *measure the distribution*. Every entry below does. A shuffle that looks fine and is 40% off uniform is the most common randomization bug there is, and you will never catch it by reading the code.

C++ Toolkit for This Module

Language material from Weiss, ***Data Structures and Algorithm Analysis in C++****, 4th ed., ch. 1 (and §10.4.1 on random number generation).*

1. Never use `rand()`

```
int badDie() { return rand() % 6 + 1; }   // two bugs in nine characters
```

- **`rand()` has a small, implementation-defined period and poor low-order bits.** On some platforms `RAND_MAX` is only 32767, so `rand()` cannot even produce every value of a 100 000-element array index.
- **`% 6` is biased.** If `RAND_MAX + 1` is not a multiple of 6, the low remainders occur more often. With `RAND_MAX = 32767`, values 1 and 2 come up measurably more than 5 and 6.

The C++11 answer separates the two concerns — an **engine** that produces raw bits, and a **distribution** that maps them to the range you want, without bias:

```
mt19937& rng() {
    // `static` inside a function: constructed ONCE, on first call,
and shared
    // by every later call. Seeding a fresh engine per call is a
classic bug --
    // it makes consecutive calls correlated, or identical.
    static mt19937 gen(random_device{}());
    return gen;
}

int randomInt(int lo, int hi) {                 // uniform on the
CLOSED range [a, b]
    return uniform_int_distribution<int>(lo, hi)(rng());
}
```

`mt19937` is the Mersenne Twister, 32-bit; `mt19937_64` for 64-bit values. `random_device{}()` provides a non-deterministic seed. **For reproducible tests, seed with a constant instead** (`static mt19937 gen(12345);`) — every measurement in this module's appendix was produced with a fixed seed so it can be re-run.

2. `uniform_int_distribution` is inclusive on both ends

`uniform_int_distribution<int>(a, b)` generates values in `[a, b]` — **`b` included**. That is unusual: almost everything else in C++ is half-open. It happens to match CLRS's `RANDOM(a, b)` exactly, which is why the translations below are one-to-one.

3. `std::shuffle`, and the function that was removed

```
void shuffleDemo(vector<int>& items) {
    shuffle(items.begin(), items.end(), rng());   // C++11 and
later: correct
    // random_shuffle(v.begin(), v.end()); // REMOVED in C++17 --
used rand() internally
}
```

`std::random_shuffle` was deprecated in C++14 and **removed in C++17** precisely because it was tied to `rand()`. `std::shuffle` takes the engine explicitly. Internally it is Fisher–Yates — the same loop as `RANDOMLY-PERMUTE`.

4. `std::set` vs `std::unordered_set` for `RANDOM-SAMPLE`

`RANDOM-SAMPLE` needs "is `i` already in `S`?" and "add `i` to `S`":

	`SET<INT>`	`UNORDERED_SET<INT>`
`count` / `insert`	`O(lg n)`	`O(1)` expected
iteration order	**sorted**	unspecified
worst case	`O(lg n)` guaranteed	`O(n)` on adversarial keys (M07)

The appendix uses `set` because the *sorted* output makes the uniformity test readable and lets a `set<int>` be a map key. In production, `unordered_set` is the right default. Note that both are heavyweight for `m` small: a `vector<int>` with a linear scan beats both for `m ≤ 30`.

5. Returning a container by value is free

```
set<int> randomSample(int sampleSize, int universe);   // returns
by value -- and that is correct
```

Weiss [§1.5.4, p.29]: in C++11 the returned container is **moved**, not copied — *"little more than a pointer change."* Do not contort this into an out-parameter. The one place it still matters is a **recursive** function that returns a container: `randomSample(m-1, n-1)` builds a fresh `set` at every level, so the recursion allocates `Θ(m)` sets. The iterative version below allocates one. That is a real difference, and it is about allocation, not about copying.

6. `numeric_limits`, not `INT_MIN`

```
int bestScore = numeric_limits<int>::min();   // <limits>, type-safe, works in templates
```

`INT_MIN` is a macro that only works for `int`. `numeric_limits<T>::min()` works for any `T` — but beware: for **floating-point** types `min()` is the smallest *positive* normal value, not the most negative. For "minus infinity" on a `double` you want `numeric_limits<double>::lowest()` or `-numeric_limits<double>::infinity()`.

Appendix — C++ for Every Pseudocode Block

Shared prelude. Every entry below uses these two helpers, which are CLRS's `RANDOM(a, b)` in C++:

```
// One engine for the whole program (toolkit 1). Swap random_device{}() for a
// literal seed when you want reproducible runs -- every measurement quoted in
// this appendix was produced with a fixed seed.
mt19937& rng() {
    static mt19937 gen(random_device{}());
    return gen;
}

// CLRS's RANDOM(a, b): a uniform integer in the CLOSED range [a, b].
int randomInt(int lo, int hi) {
    return uniform_int_distribution<int>(lo, hi)(rng());
}
```

A1 HIRE-ASSISTANT

Pseudocode: §2, "Problem".

```
// `rank` is 1-indexed to match CLRS: rank[0] is the dummy candidate "0", who is
// less qualified than everyone, and rank[1..n] are the real candidates.
```

```
// Returns the NUMBER OF HIRES -- the quantity the analysis is
about. The cost
// model is c_i * n interviews + c_h * hires, and only `hires`
varies.
int hireAssistant(const vector<int>& quality) {
    const int n = (int)quality.size() - 1;      // -1 for the dummy
at index 0
    int bestSoFar = 0;                           // 1  best = 0
    int hireCount = 0;
    for (int candidate = 1; candidate <= n; ++candidate) {
  // 2  for i = 1 to n
        if (quality[candidate] > quality[bestSoFar]) {          //
4  if candidate i is better than best
            bestSoFar = candidate;                          // 5
best = i
            ++hireCount;                          // 6      hire
candidate i
        }
    }
    return hireCount;
}
```

Complexity. $\Theta(n)$ interviews always. Hires: n **in the worst case** (candidates arrive in increasing order of quality — you hire every one), **1 in the best case** (decreasing order). The *worst case is entirely a property of the input order*, which is exactly what randomization will take away from the adversary.

> *Verified: increasing input → **10 hires** for `n = 10`; decreasing input → **1 hire**.*

A2 RANDOMIZED-HIRE-ASSISTANT and A3 RANDOMLY-PERMUTE

Pseudocode: §2, "The randomized version" and §3, "Algorithm".

```
// Fisher-Yates. The ONE thing to get right is the range of the
random draw.
void randomlyPermute(vector<int>& arr) {
    const int n = (int)arr.size() - 1;          // 1-indexed, A[0]
unused
    for (int slot = 1; slot <= n; ++slot)
        // RANDOM(i, n)  -- from i to n, NOT from 1 to n.
        //
```

```
        // WHY: the loop invariant is "after iteration i, A[1..i]
is a uniformly
        // random i-permutation of a uniformly random i-subset".
Drawing from
        // [i, n] means position i is filled from the elements NOT
YET PLACED,
        // which keeps every arrangement equally likely.
        //
        // Drawing from [1, n] instead gives n^n equally likely
execution paths
        // mapped onto n! permutations -- and n! does not divide
n^n for n >= 3,
        // so SOME permutation must be more likely than another. It
is not a
        // subtle bias; see the measurement below.
        //
        // swap(A[i], A[i]) when the draw returns i is intentional
and required:
        // "leave it where it is" must be one of the possible
outcomes.
        swap(arr[slot], arr[randomInt(slot, n)]);
}

// The whole randomized algorithm: shuffle, then run the
deterministic one.
// Takes `rank` BY VALUE because it must permute it -- the caller's
order is
// preserved, and a caller passing a temporary gets a move, not a
copy.
int randomizedHireAssistant(vector<int> quality) {
    randomlyPermute(quality);                   // 1  randomly
permute the candidates
    return hireAssistant(quality);              // 2  HIRE-
ASSISTANT(n)
}
```

Complexity. `Θ(n)` for the permute, `Θ(n)` for the hiring pass.

Expected hires = `H_n = ln n + O(1)`. By linearity of expectation over indicator variables `X_i = I{candidate i is hired}`: candidate `i` is hired iff it is the best of the first `i`, which happens with probability `1/i` under a uniform random order. So `E[hires] = Σ_i 1/i = H_n`. **This is a guarantee about the algorithm, not an assumption about the input** — the adversary may hand you the worst possible list and the bound still holds.

> *Verified (2000 trials per row, on the adversarial increasing input):*
>
> | *N* | *MEASURED HIRES* | *H_N* | *RATIO* |
> |---|---|---|---|
> | *10* | *2.908* | *2.929* | *0.993* |
> | *100* | *5.183* | *5.187* | *0.999* |
> | *1 000* | *7.508* | *7.485* | *1.003* |
> | *10 000* | *9.695* | *9.788* | *0.991* |
>
> ***And the uniformity of the shuffle itself**, over 240 000 trials on `n = 4` (all 24 permutations):*
>
> | *VERSION* | *WORST DEVIATION FROM UNIFORM* |
> |---|---|
> | `swap(A[i], A[RANDOM(i, n)])` *— correct* | ***1.91%*** |
> | `swap(A[i], A[RANDOM(1, n)])` *— the classic bug* | ***39.65%*** |
>
> *Both produce a plausible-looking shuffle. Only one of them is uniform. `4^4 = 256` execution paths cannot divide evenly among `4! = 24` permutations, and 39.65% is what that looks like.*

A4 RANDOM-SAMPLE

Pseudocode: §3, *"Related: sampling `m` of `n`".*

```
// LITERAL recursive translation of CLRS's RANDOM-SAMPLE.
//
// The beautiful part: it never needs to know which elements are
already chosen
// in order to avoid them. If the draw collides with something
already in S, it
// adds `n` instead -- and `n` is guaranteed not to be in S,
because S was built
```

```
// from RANDOM-SAMPLE(m-1, n-1), whose values are all <= n-1.
set<int> randomSample(int sampleSize, int universe) {
    if (sampleSize == 0) return {};                    // 1  S =
empty
    set<int> sample = randomSample(sampleSize - 1, universe - 1);
 //    recurse on the smaller problem
    int draw = randomInt(1, universe);                   // 3  i =
RANDOM(1, k)
    if (sample.count(draw)) sample.insert(universe);
 // 4  if i in S: S = S + {k}
    else            sample.insert(draw);               // 5  else:
   S = S + {i}
    return sample;                                   // returned BY
VALUE -- moved, not copied
}

// The same thing as a loop, which is what you should actually
write: one `set`
// instead of m of them, and no Theta(m) stack depth.
set<int> randomSampleIterative(int sampleSize, int universe) {
    set<int> sample;
    for (int upperBound = universe - sampleSize + 1; upperBound <=
universe; ++upperBound) {    // 2  for k = n-m+1 to n  (m
iterations)
        int draw = randomInt(1, upperBound);
        if (sample.count(draw)) sample.insert(upperBound);
        else            sample.insert(draw);
    }
    return sample;
}
```

Complexity. $\Theta(m \lg m)$ with `set` (m insertions into a tree of size $\le m$), $\Theta(m)$ expected with `unordered_set`. **Space $\Theta(m)$ — independent of n.** That is the point: you can sample 10 items from 10^{18} without materialising anything.

Why every m-subset is equally likely is a neat induction: by hypothesis S after the recursive call is a uniform $(m-1)$-subset of $\{1..n-1\}$. Then each of the n possible draws maps to a distinct outcome, and the "collision → add n" branch is precisely what makes every subset containing n as likely as every subset not containing it.

> *Verified:* `randomSampleIterative(3, 8)` *over 200 000 trials produced* ***all*** `C(8,3) = 56` ***subsets****, with a worst-case deviation from uniform of* ***4.27%****. Both forms always returned exactly* `m` *distinct values in* `[1, n]`*.*

A5 ONLINE-MAXIMUM (the secretary problem)

Pseudocode: §6, "The Secretary Problem".

```
// Observe the first k candidates without hiring; then hire the
first one that
// beats everything seen so far. Return n if nobody does (you are
stuck with the
// last candidate).
int onlineMaximum(const vector<int>& scores, int observeCount) {
    const int n = (int)scores.size() - 1;
    // numeric_limits, not INT_MIN (toolkit 6). Careful: for a
double-valued
    // score you would need lowest(), not min().
    int bestObserved = numeric_limits<int>::min();
    for (int candidate = 1; candidate <= observeCount; ++candidate)
                    // 2  observation phase
        if (scores[candidate] > bestObserved) bestObserved =
scores[candidate];
    for (int candidate = observeCount + 1; candidate <= n;
++candidate)                    // 4  selection phase
        if (scores[candidate] > bestObserved) return candidate;
       // 5      commit, irrevocably
    return n;                                          // 6  no one
qualified
}

// The online cousin, worth knowing because it is the one
interviewers ask for:
// reservoir sampling picks a UNIFORMLY RANDOM element from a
stream of unknown
// length, in O(1) space, in one pass.
//
// Why it is uniform: element t is kept with probability 1/t, and
then survives
// every later step with probability (t/(t+1)) * ((t+1)/(t+2)) *
... * ((n-1)/n),
```

```
// which telescopes to t/n. Multiply: (1/t) * (t/n) = 1/n. Every
element, same.
int reservoirPick(const vector<int>& items) {
    int kept = -1, seenCount = 0;
    for (int item : items) {
        ++seenCount;
        if (randomInt(1, seenCount) == 1) kept = item;
    }
    return kept;
}
```

Complexity. `Θ(n)` time, `Θ(1)` space, and — crucially — **one pass with no take-backs**. This is the defining shape of an *online* algorithm (M24 *(planned)*).

The analysis. With `k = n/e`, the probability of ending up with the genuine best candidate tends to `1/e ≈ 0.368`. Intuition: `k` too small and you commit before you know what "good" looks like; `k` too large and the best candidate has probably already gone by. The optimum balances the two, and remarkably the answer does not vanish as `n → ∞` — you keep a **constant** 37% chance no matter how many candidates there are.

> *Verified: `n = 100`, 10 000 trials per `k`:*
>
> | *k* | *1* | *10* | *20* | *30* | *37* | *50* | *70* | *99* |
> |---|---|---|---|---|---|---|---|---|
> | *P(pick the best)* | *0.053* | *0.233* | *0.328* | *0.366* | ***0.373*** | *0.342* | *0.253* | *0.012* |
>
> *The measured optimum is `k = 37` against the predicted `n/e = 36.8`, with success probability **0.3729** against `1/e = 0.3679`. Reservoir sampling over a 6-element stream was uniform to within **0.97%** over 120 000 trials.*

Previous: M03 — Divide & Conquer and Recurrences · Next: M05 — Sorting & Order Statistics